BÁO CÁO THỰC HÀNH
KỸ THUẬT DỊCH NGƯỢC GAME ANDROID

Phân tích tài nguyên Unity và các luồng xử lý coin trong SandSort

Sinh viên: Nguyễn Quang Minh

Mã sinh viên: 24CQGA098

Lớp: B24CQGA02-B

Ngày báo cáo: 11/10/2026

1. Mục tiêu và phạm vi

Bài thực hành khảo sát APK SandSort, xuất một tài nguyên hình ảnh và xác định hai luồng chính: nạp hoặc khởi tạo coin lúc vào game; cập nhật coin trong nhánh thưởng sau khi hoàn thành level. Phần khảo sát bổ sung phân biệt thưởng thường, quảng cáo X2 và thưởng trong RewardManager, đồng thời làm rõ các cờ kiểm soát thao tác nhận thưởng.

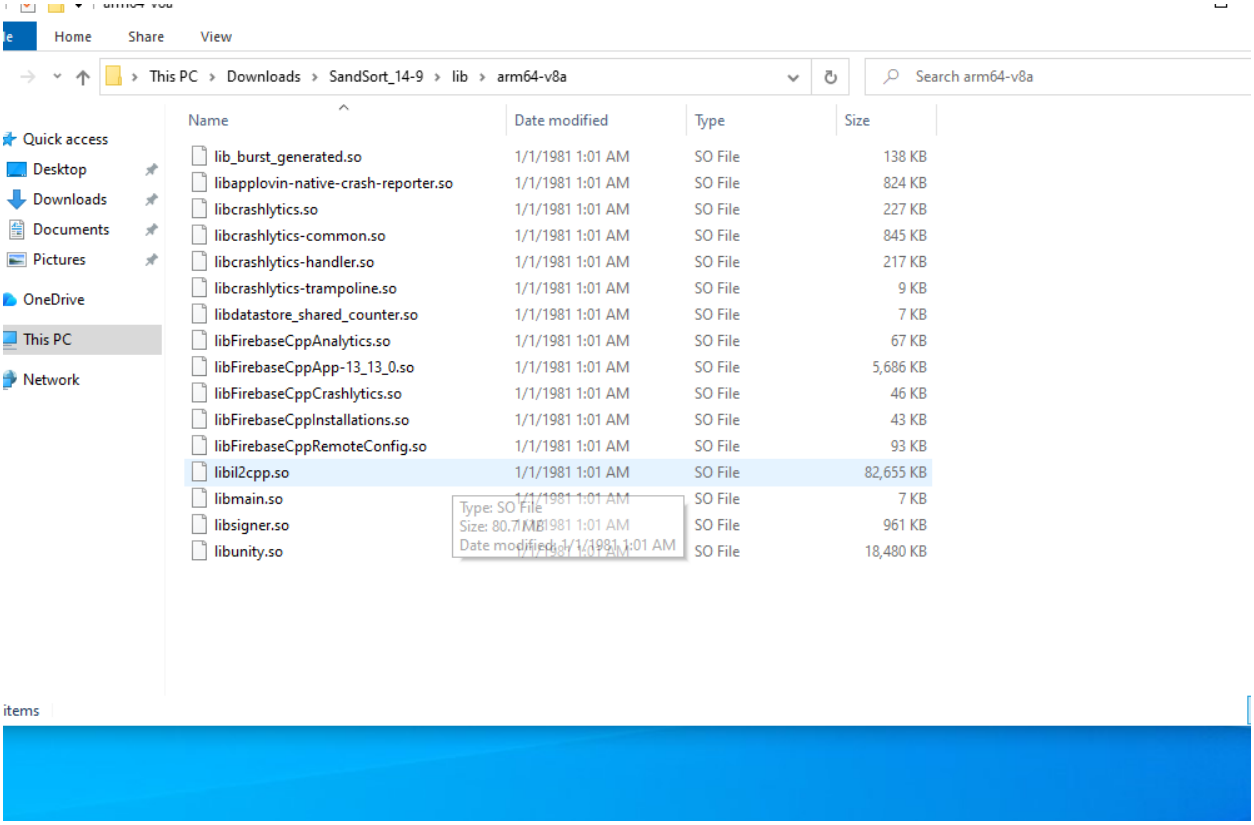
Quy trình sử dụng UABEA, Il2CppDumper, dnSpy và Ghidra. Mỗi kết luận về hành vi phải nối được tên phương thức, địa chỉ mã native, offset trường hoặc lời gọi thực thi; tên hàm và thân rỗng trong DLL giả chỉ được dùng để tìm ứng viên. Báo cáo dựa trên phân tích tĩnh và 27 ảnh thực hành, chưa có kết quả sửa APK hoặc đo số dư khi game chạy.

2. Môi trường, dữ liệu đầu vào và công cụ

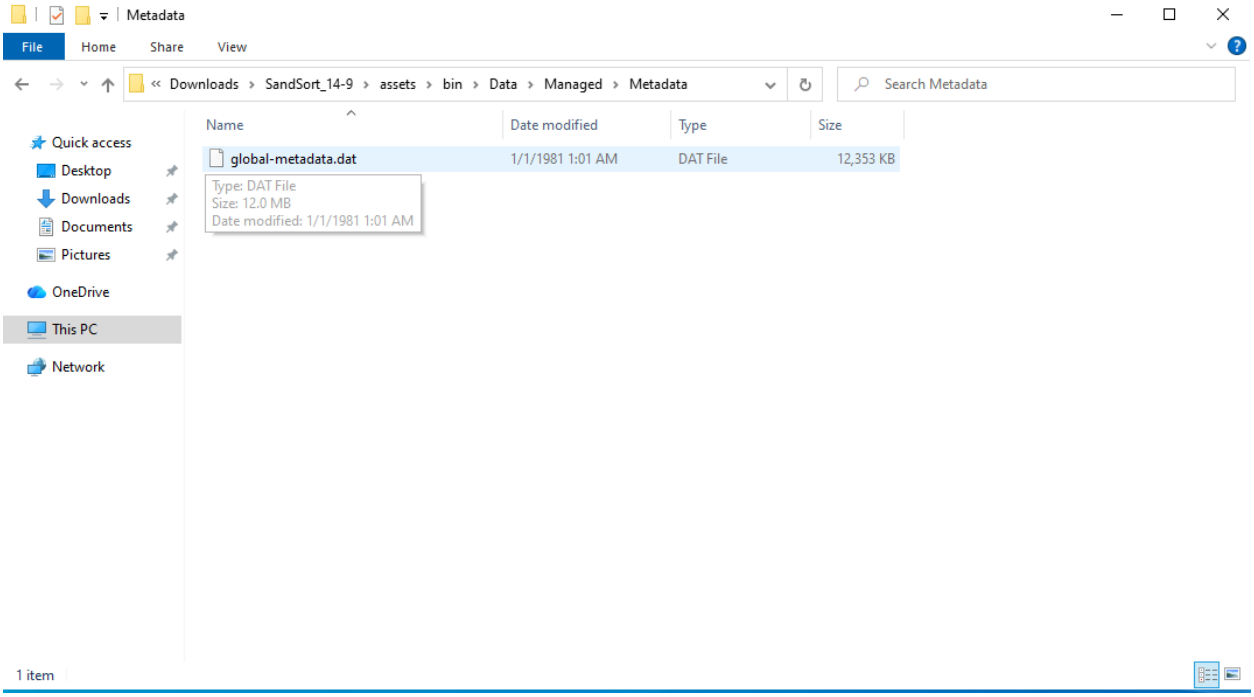
2.1. Mẫu phân tích

Tên game theo bài thực hành là SandSort; các lớp được quan sát thuộc namespace FoodSort. Kiến trúc được chọn xuyên suốt là ARM64 (arm64-v8a). Tên file APK đầy đủ, phiên bản hoặc mã nhận diện bản APK chưa được cung cấp, nên cần bổ sung trước khi nộp để người chấm đối chiếu đúng mẫu.

Thư viện native nằm tại lib/arm64-v8a/libil2cpp.so. Metadata tương ứng nằm tại assets/bin/Data/Managed/Metadata/global-metadata.dat. Hai file phải lấy từ cùng APK; thư viện ARM64 dùng với Il2CppDumper cũng là thư viện import vào Ghidra.



Hình 1. Thư viện libil2cpp.so ARM64 trong APK đã giải nén.



Hình 2. File global-metadata.dat dùng cùng thư viện native.

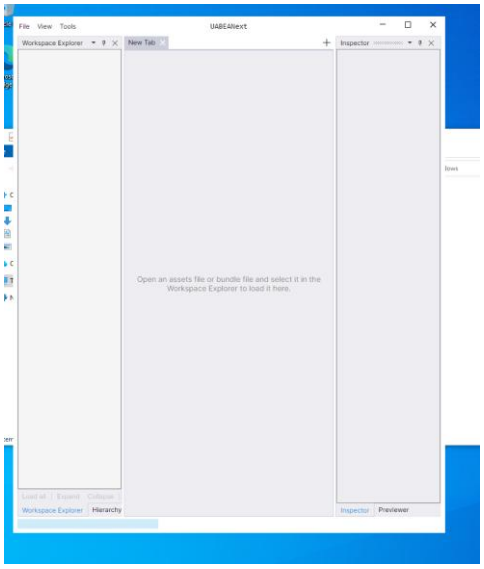
2.2. Công cụ và trạng thái sử dụng

Các ảnh thể hiện môi trường Windows. UABEANext đã khởi chạy và được dùng xuất asset; Il2CppDumper đã hoàn tất dump; dnSpy đã mở Assembly-CSharp.dll; Ghidra đã phân tích libil2cpp.so và hiển thị tên phương thức. Đường dẫn ở Hình 6 cho thấy gói Il2CppDumper-win-v6.7.46; phiên bản các công cụ còn lại và cấu hình VMware chưa được ghi nhận trong bộ ảnh.

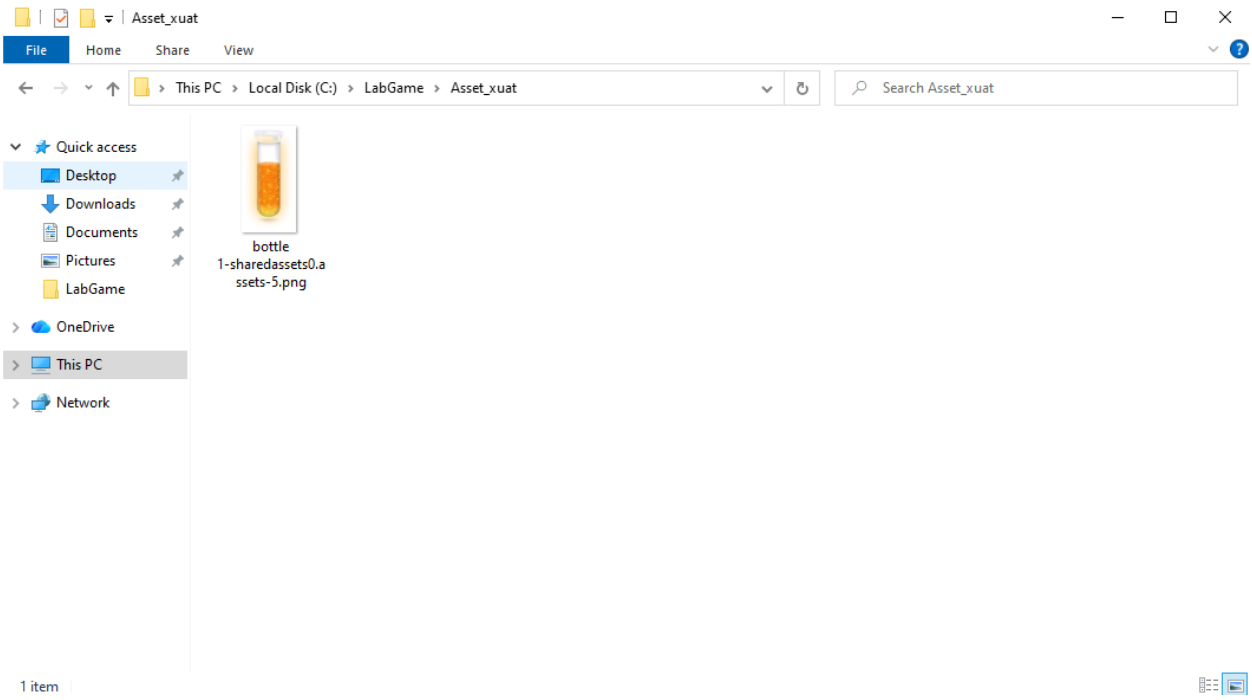
3. Xem và xuất tài nguyên Unity

3.1. Thao tác và kết quả

UABEA xử lý dữ liệu Unity dưới assets/bin/Data. Trong lần thực hành này, file chứa asset là sharedassets0.assets và tên được ghi nhận là bottle 1. Hình 3 chứng minh ứng dụng mở được; Hình 4 và 5 thể hiện file đã xuất và hình ảnh kết quả. Ảnh cửa sổ ban đầu chưa phải bằng chứng đã chọn đúng loại asset.



Hình 3. UABEANext khởi chạy và sẵn sàng mở tài nguyên Unity.



Hình 4. File PNG bottle 1 đã xuất từ sharedassets0.assets.



Hình 5. Hình ảnh bottle 1 sau khi xuất sang PNG.

3.2. Nhận xét về tài nguyên

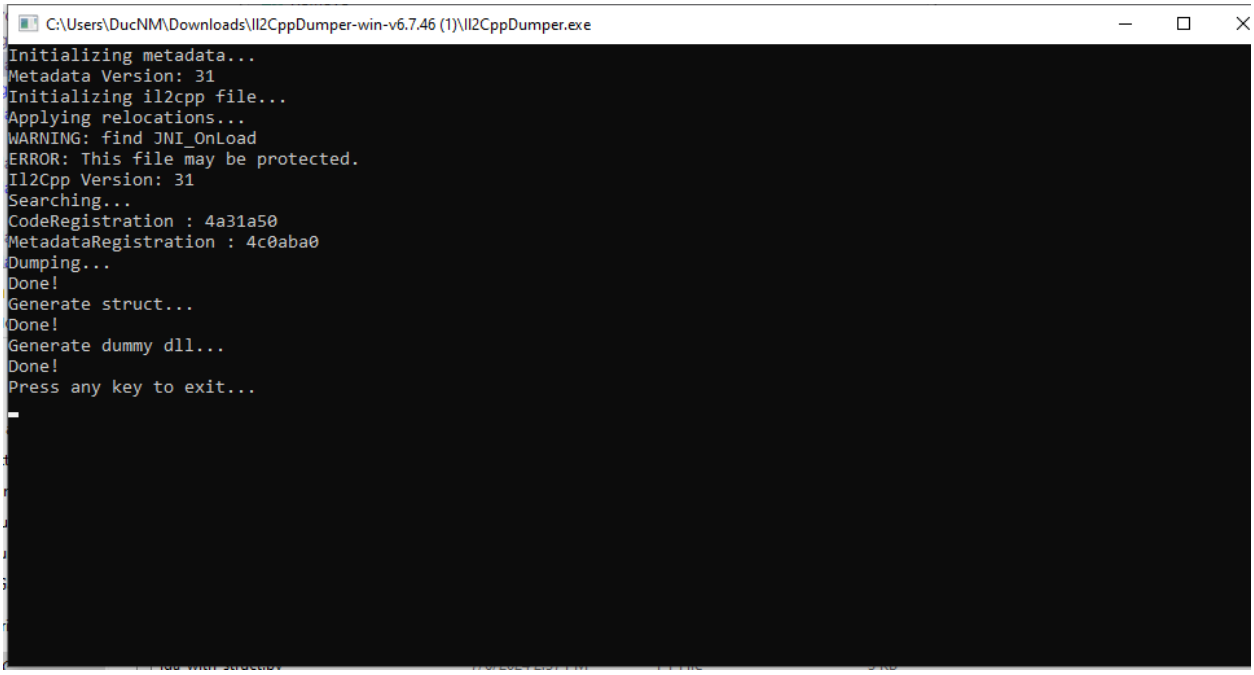
Ảnh PNG hiển thị một vật thể dạng chai hoặc ống chứa các hạt màu vàng. Tài nguyên có thể dùng để biểu diễn vật chứa trong game sắp xếp; vai trò chính xác cần đối chiếu giao diện game đang chạy. Loại asset trong cột Type của UABEA chưa được ghi lại rõ trong bộ ảnh, nên không khẳng định Texture2D hay Sprite chỉ từ file PNG.

Đường dẫn dữ liệu Unity cần ghi trong hồ sơ là assets/bin/Data/sharedassets0.assets; file PNG đã xuất phải được giữ cùng thư mục kết quả. Bộ ảnh hiện có chứng minh kết quả xuất, nhưng chưa có ảnh riêng của Preview và cột Type trong UABEA. Việc thao tác asset đồ họa không cung cấp bằng chứng về logic cộng coin.

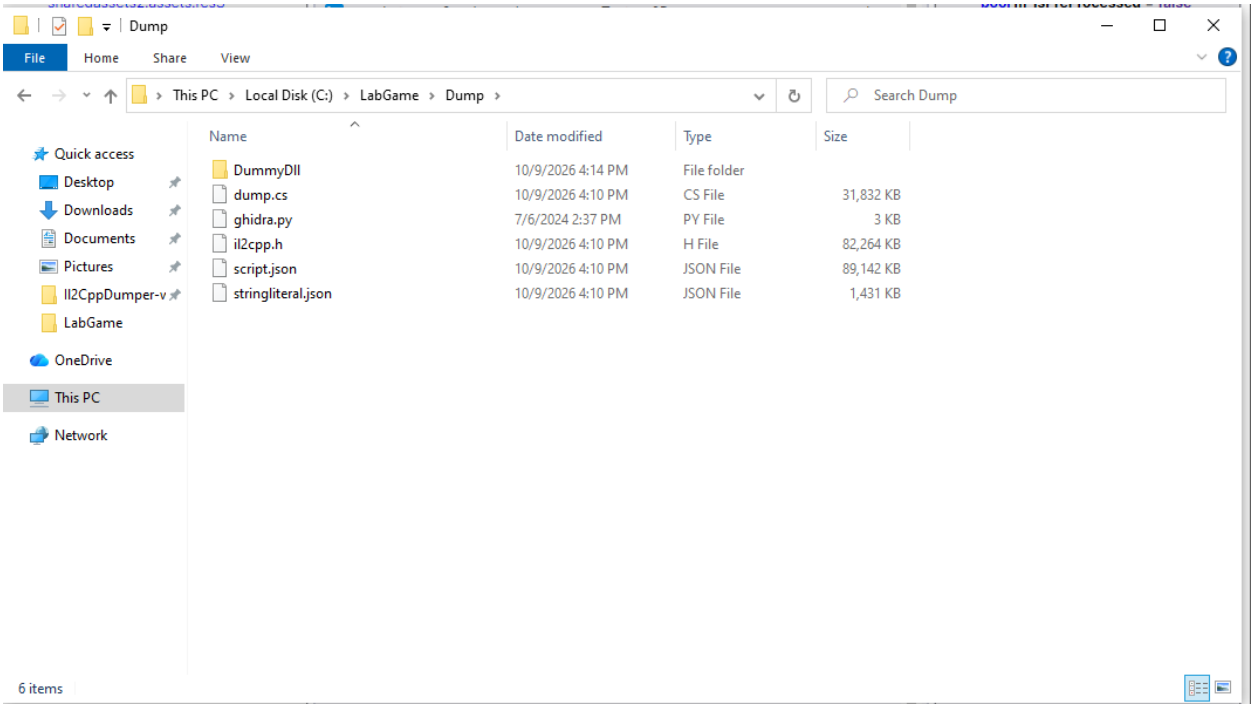
4. Tra cứu cấu trúc và địa chỉ IL2CPP

4.1. Đầu vào và kết quả Il2CppDumper

Il2CppDumper kết hợp libil2cpp.so ARM64 với global-metadata.dat để dựng dữ liệu tra cứu. Hình 6 có thông báo cảnh báo về thư viện, nhưng phía sau vẫn xuất hiện các bước Dumping, Generate struct, Generate dummy dll và Done. Vì vậy, đánh giá kết quả phải dựa trên đầu ra thực tế và việc đối chiếu thành công, không chỉ một dòng cảnh báo.



Hình 6. Il2CppDumper hoàn tất dump và tạo DLL giả.

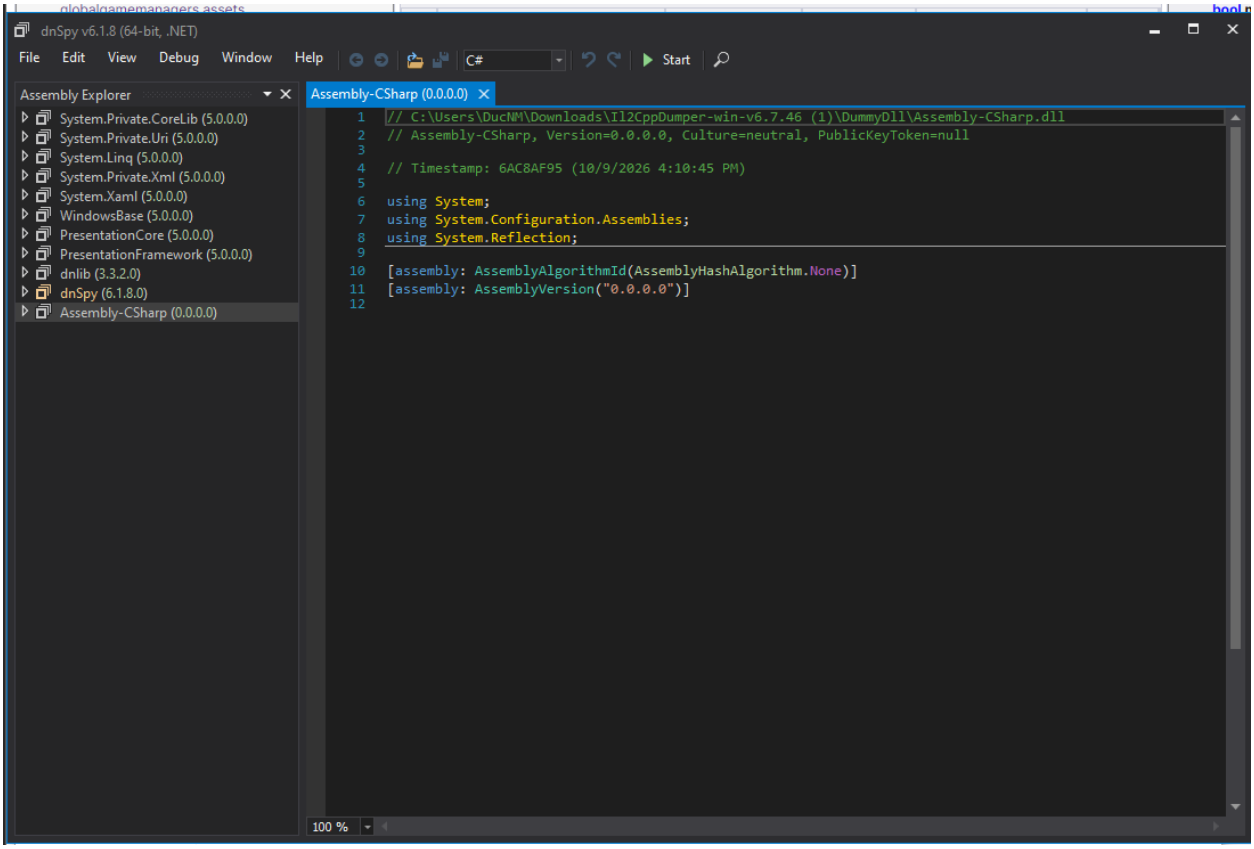


Hình 7. Các đầu ra dump.cs, DummyDll, script.json và ghidra.py.

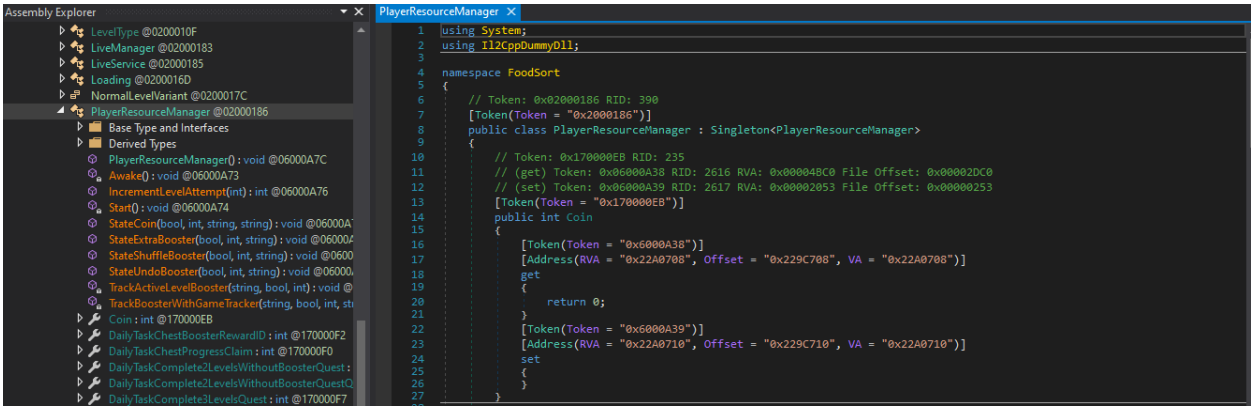
dump.cs cung cấp tên lớp, chữ ký phương thức, offset trường và RVA. DummyDll giúp dnSpy hiển thị cấu trúc C#; script.json và ghidra.py hỗ trợ gắn tên phương thức trong Ghidra. Các đầu ra ở Hình 7 được giữ từ cùng lần dump để tránh nhầm giữa phiên bản APK hoặc kiến trúc.

4.2. Dùng dnSpy tìm ứng viên

Trong dnSpy, Assembly-CSharp.dll thuộc DummyDll được mở để khảo sát các lớp quản lý tài nguyên, phần thưởng và màn hình thắng. Các ứng viên chính là PlayerResourceManager, RewardLevelWinObserver, RewardManager và UIWinGame. Đây là bước lập bản đồ trước khi đọc mã native.

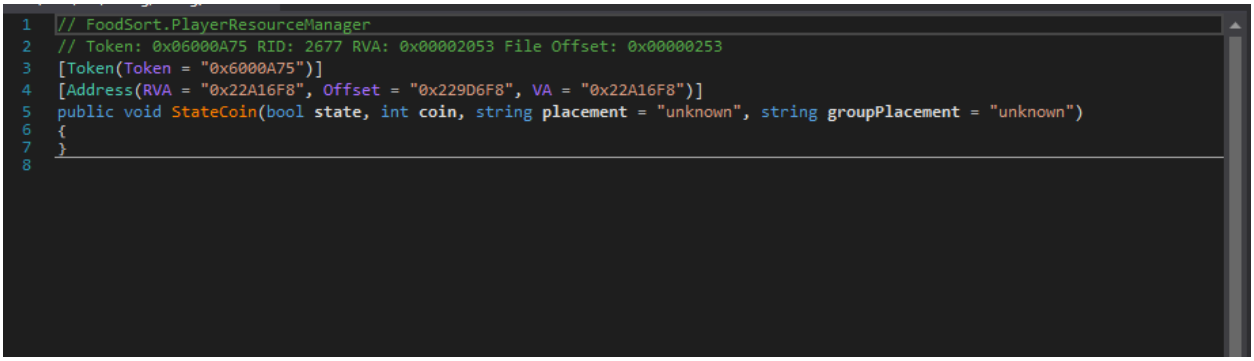


Hình 8. Assembly-CSharp.dll được mở bằng dnSpy.



Hình 9. Thuộc tính Coin và RVA của getter, setter.

Trong PlayerResourceManager, thuộc tính Coin có getter RVA 0x22A0708 và setter RVA 0x22A0710. Phần C# giả của getter trả về 0 và setter rỗng không có nghĩa game luôn có 0 coin; hành vi thực tế phải kiểm tra trong Ghidra. Hai địa chỉ này được dùng để tìm nơi đọc, ghi số dư.



```
// FoodSort.PlayerResourceManager
// Token: 0x06000A75 RID: 2677 RVA: 0x00002053 File Offset: 0x00000253
[Token(Token = "0x6000A75")]
[Address(RVA = "0x22A16F8", Offset = "0x229D6F8", VA = "0x22A16F8")]
public void StateCoin(bool state, int coin, string placement = "unknown", string groupPlacement = "unknown")
{
}
```

Hình 10. Chữ ký StateCoin trong DLL giả.

StateCoin có chữ ký StateCoin(bool state, int coin, string placement, string groupPlacement) và RVA 0x22A16F8. Chữ ký cho biết các tham số cần theo dõi, nhưng chưa chứng minh state chọn cộng hay trừ. Start có RVA 0x22A15D4 là ứng viên nạp dữ liệu lúc khởi động; Awake tại 0x22A15D0 được loại trừ sau khi thấy chỉ có lệnh ret.

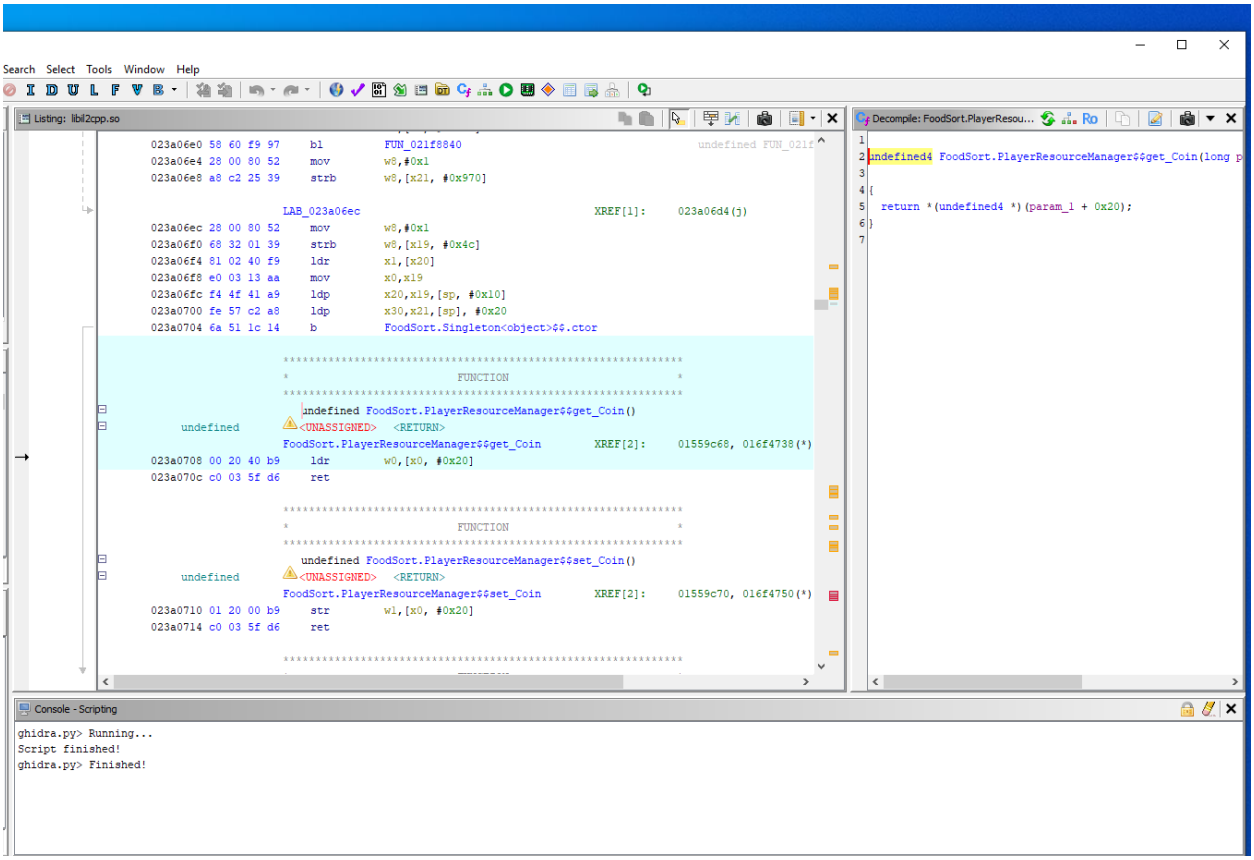
4.3. Đối chiếu địa chỉ trong Ghidra

Project Ghidra sử dụng Image Base 0x100000. Địa chỉ tra cứu bằng RVA + 0x100000; ví dụ StateCoin có RVA 0x22A16F8 và địa chỉ Ghidra 0x023A16F8. RVA, địa chỉ hiển thị trong Ghidra, File Offset và offset trường trong đối tượng là các khái niệm khác nhau.

Listing thể hiện lệnh ARM64 cùng địa chỉ; Decompiler dựng mã giả để đọc nhánh và lời gọi; Symbol Tree hỗ trợ tìm tên đã gắn bằng script. Mã giả không phải mã C++ gốc. Các cảnh báo về nhảy gián tiếp hoặc kiểu dữ liệu thiếu được xem là giới hạn của công cụ, không được bỏ qua khi suy luận.

5. Trường coin và luồng nạp khi vào game

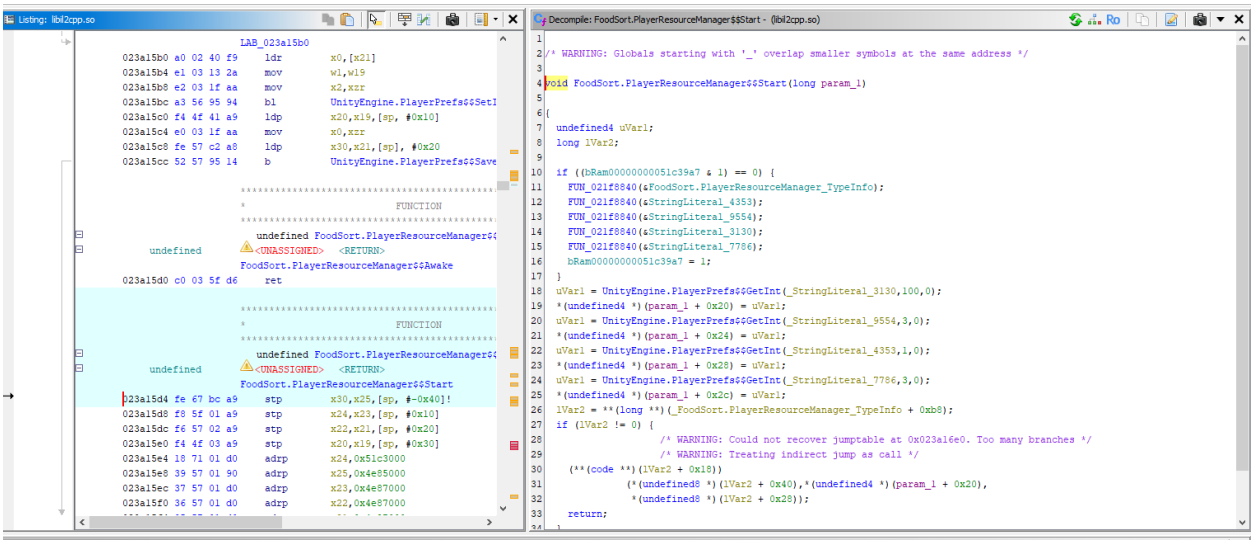
5.1. Xác định nơi đọc và ghi coin



Hình 11. LDR và STR truy cập coin tại offset 0x20 trong Ghidra.

get_Coin tại 0x023A0708 dùng LDR để đọc giá trị ở offset 0x20 của đối tượng. set_Coin tại 0x023A0710 dùng STR để ghi vào cùng vị trí. Kết quả này xác định trường số dư đang được sử dụng trong PlayerResourceManager; offset 0x20 là vị trí trong đối tượng, không phải vị trí trong file APK.

5.2. Start nạp dữ liệu đã lưu



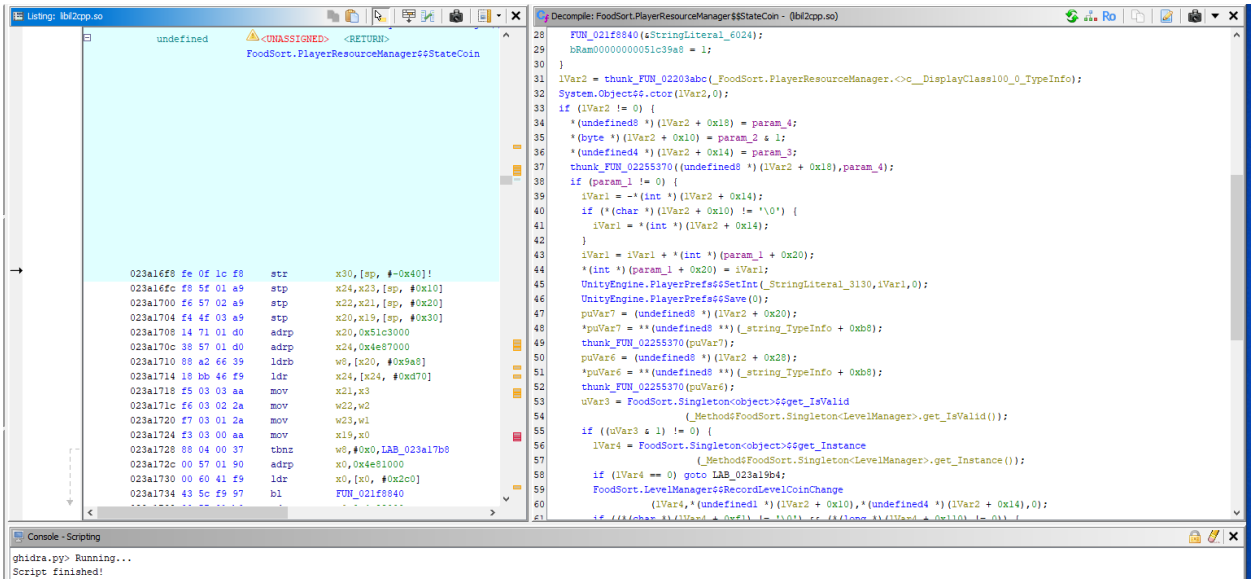
Hình 12. Start đọc PlayerPrefs với mặc định 100 và ghi coin vào đối tượng.

Start tại 0x023A15D4 gọi PlayerPrefs.GetInt(StringLiteral_3130, 100) rồi ghi kết quả vào offset 0x20. Nếu khóa chưa tồn tại, giá trị được dùng là 100; nếu đã có dữ liệu lưu, hàm lấy số đã lưu. Do đó, 100 là giá trị mặc định, không phải lượng được cộng thêm mỗi lần mở game.

Các lời gọi GetInt tiếp theo có mặc định 3, 1, 3 và ghi vào offset 0x24, 0x28, 0x2C. Chúng không ghi vào trường coin ở 0x20 nên không được dùng làm số coin ban đầu. StringLiteral_3130 là ký hiệu do công cụ gắn; nội dung chuỗi khóa chưa được giải mã trong lần thực hành này.

Luồng đã chứng minh là Start → đọc PlayerPrefs → ghi trường coin. Bằng chứng không chỉ dựa vào tên Start mà dựa vào nguồn dữ liệu, mặc định 100 và địa chỉ đích ghi trùng với getter, setter.

5.3. StateCoin cập nhật và lưu số dư



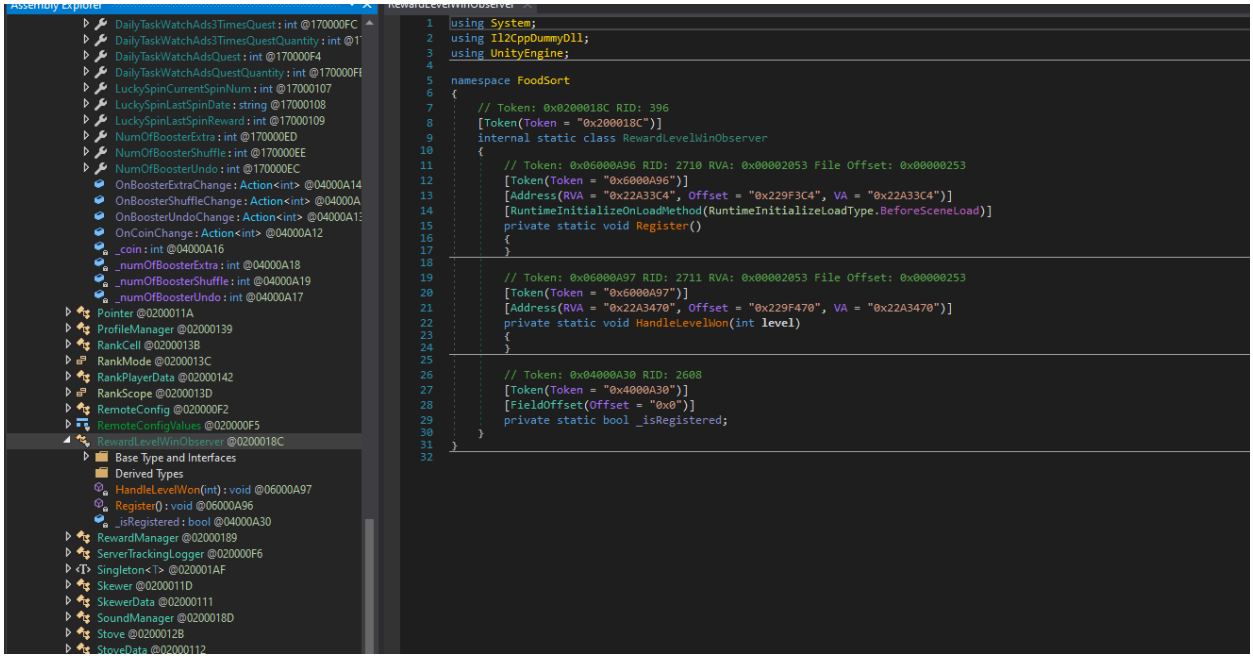
Hình 13. StateCoin chọn dấu, cộng số dư và lưu bằng PlayerPrefs.

Trong StateCoin tại 0x023A16F8, cờ và lượng coin được đưa vào một đối tượng hỗ trợ tại các offset 0x10, 0x14; đây không phải trường số dư của PlayerResourceManager. Mã chọn delta = -amount khi state = false và delta = amount khi state = true, rồi tính newCoin = oldCoin + delta.

Kết quả được ghi lại vào param_1 + 0x20. Ngay sau đó, PlayerPrefs.SetInt(StringLiteral_3130, newCoin) và PlayerPrefs.Save được gọi. Ký hiệu khóa trùng với khóa đọc trong Start, nối được luồng nạp lúc khởi động với luồng lưu sau cập nhật. Phần ảnh còn cho thấy RecordLevelCoinChange, nhưng không dùng tên lời gọi đó để kết luận có xác thực máy chủ.

6. Khảo sát nhánh RewardManager và thưởng 500 coin

6.1. Từ ứng viên đến đường đi của sự kiện



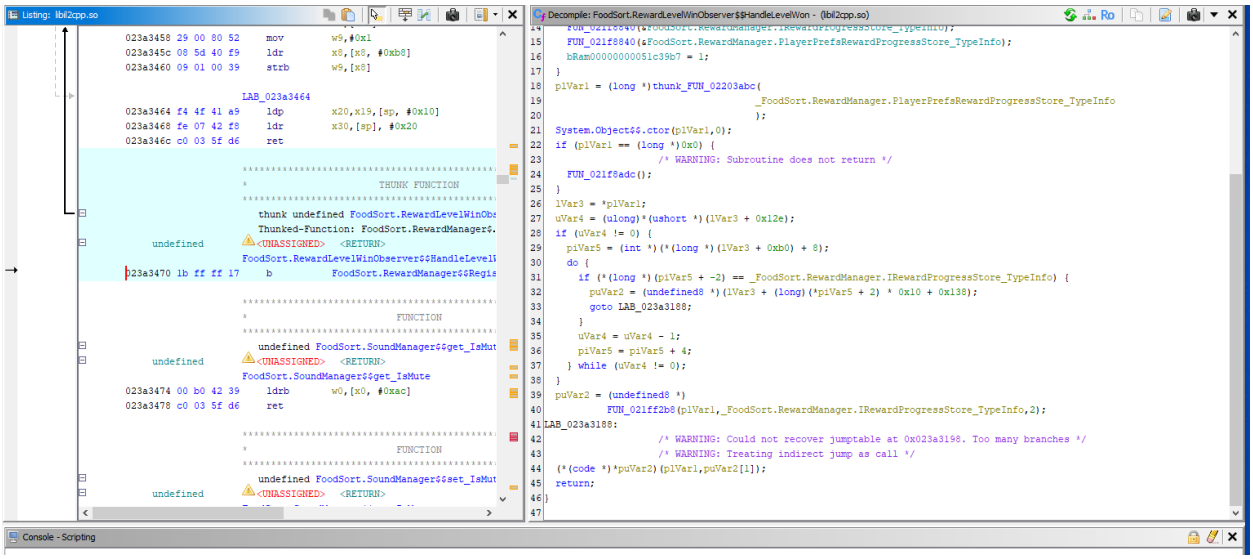
Hình 14. Các phương thức Register và HandleLevelWon của RewardLevelWinObserver.

RewardLevelWinObserver có Register tại RVA 0x22A33C4 và HandleLevelWon(int level) tại RVA 0x22A3470. Tên và chữ ký gợi ý liên quan đến sự kiện thắng level; bằng chứng đường đi được kiểm tra riêng bằng Listing, thay vì coi tên hàm là bằng chứng cộng thưởng.



Hình 15. Chữ ký và RVA của ReceiveReward trong RewardManager.

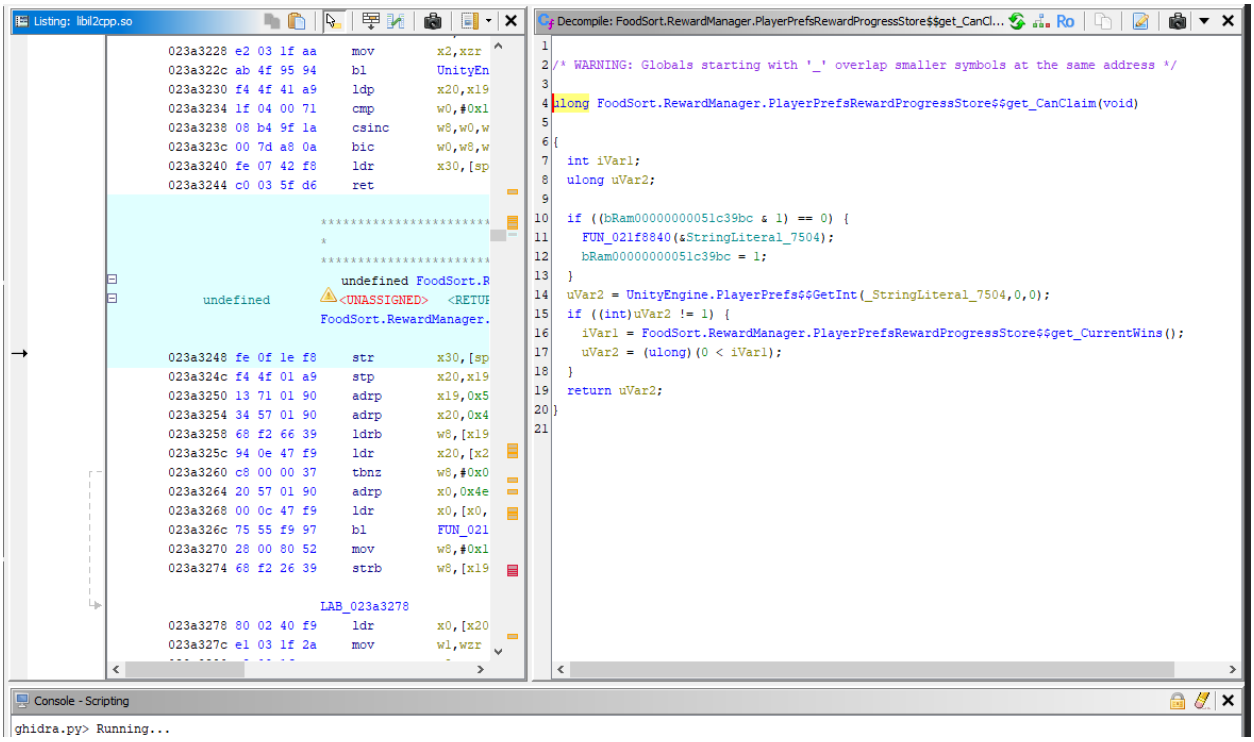
RewardManager có các ứng viên ReceiveReward (RVA 0x22A2DDC) và RegisterWinProgress (RVA 0x22A30DC). ReceiveReward được khảo sát để tìm thao tác cập nhật coin; RegisterWinProgress và kho lưu tiến độ được khảo sát để phân biệt ghi nhận chiến thắng với nhận tiền thưởng.



Hình 16. HandleLevelWon chuyển tiếp đến RegisterWinProgress.

Listing tại 0x023A3470 thể hiện lệnh B chuyển tiếp từ HandleLevelWon đến RewardManager.RegisterWinProgress. Phần Decompiler khảo sát đường đi tới kho tiến độ PlayerPrefsRewardProgressStore qua lời gọi gián tiếp. Đoạn này chứng minh chuyển tiếp xử lý sự kiện, chưa chứng minh số dư được cộng ngay tại HandleLevelWon.

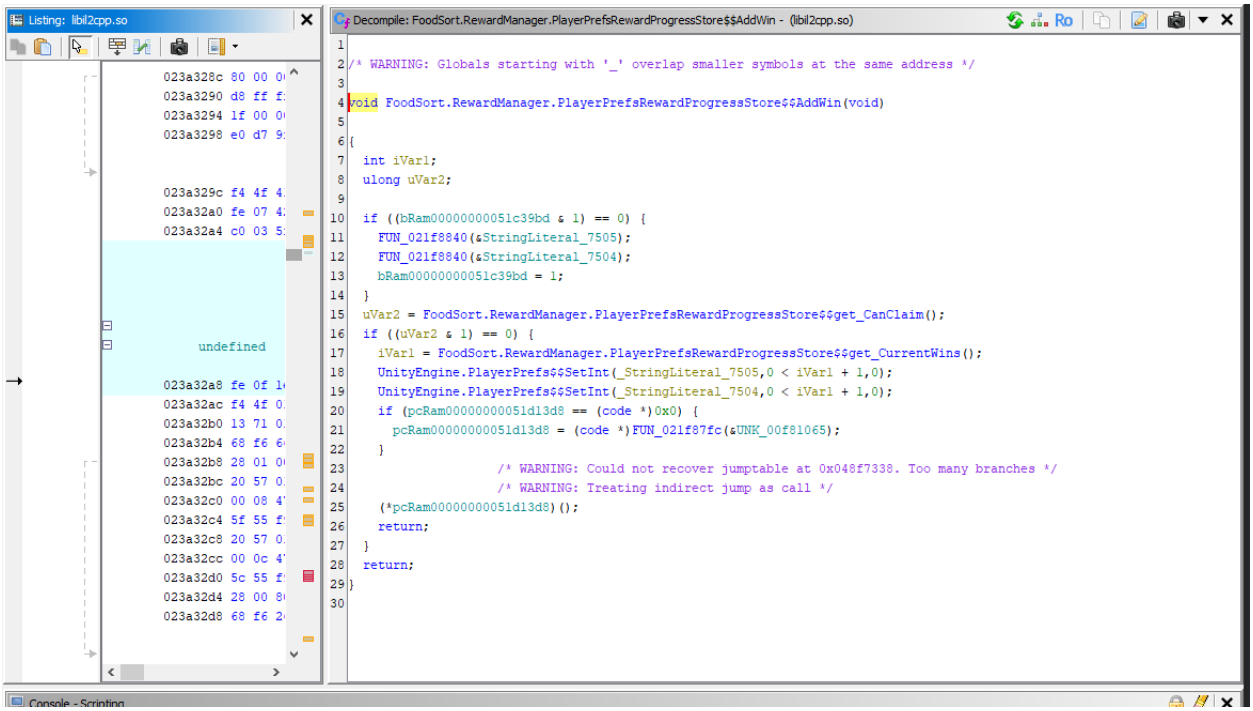
6.2. Điều kiện nhận thưởng và cập nhật tiến độ



Hình 17. get_CanClaim đọc trạng thái lưu và kiểm tra CurrWins.

get_CanClaim tại 0x023A3248 đọc PlayerPrefs.GetInt(StringLiteral_7504, 0). Nếu giá trị đọc được bằng 1, hàm trả kết quả cho phép nhận thưởng; nếu khác 1, hàm gọi get_CurrWins rồi trả về phép kiểm tra CurrWins > 0. Có thể diễn giải điều kiện quan sát được là (giá trị khóa 7504 == 1) hoặc (CurrWins > 0).

Chưa giải mã nội dung khóa nên báo cáo gọi đây là giá trị trạng thái lưu, không tự gán nghĩa là cờ đã nhận thưởng. Đoạn mã cũng không thể hiện một ngưỡng nhiều chiến thắng cố định; cần phân biệt biểu thức thực tế với suy đoán từ tên lớp.

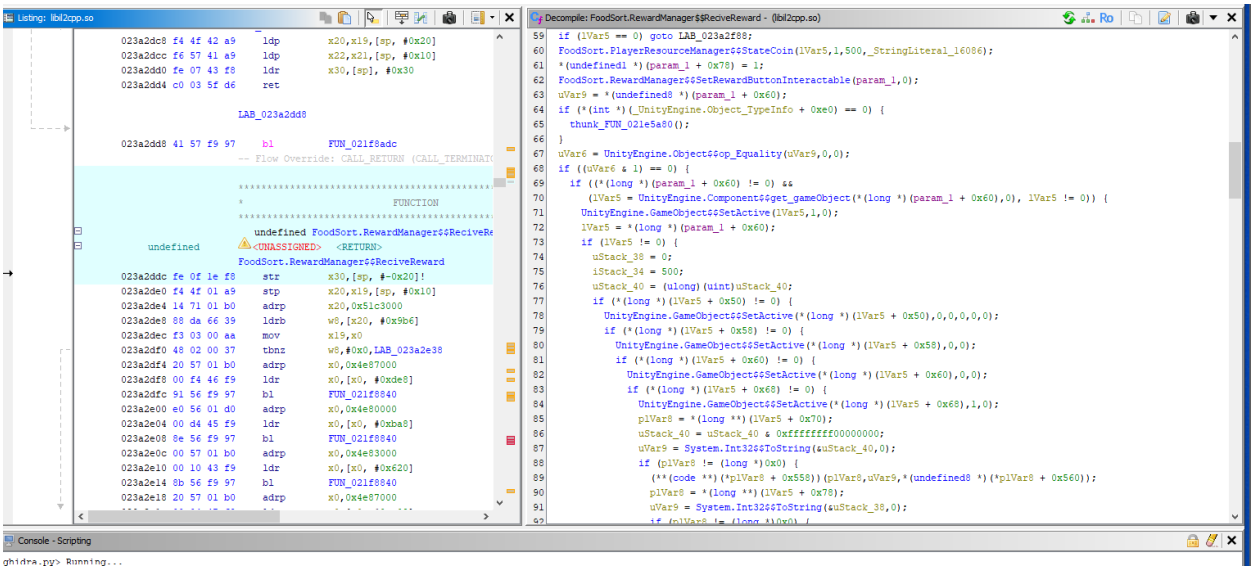


Hình 18. AddWin cập nhật tiến độ khi chưa được phép nhận thưởng.

AddWin tại 0x023A32A8 trước hết gọi get_CanClaim. Chỉ khi kết quả bằng false, hàm đọc CurrWins. Trong mã giả ở Hình 18, cả hai lời gọi SetInt cho StringLiteral_7505 và StringLiteral_7504 đều truyền biểu thức 0 < (CurrWins + 1), tức kết quả so sánh biểu diễn bằng 0 hoặc 1. Vì vậy, không kết luận từ ảnh này rằng một bộ đếm số lần thắng được tăng và lưu nguyên giá trị CurrWins + 1. Nếu đã có quyền nhận thưởng, nhánh cập nhật bị bỏ qua.

Đây là dữ liệu tiến độ và trạng thái cho phép nhận thưởng. Không có lời gọi StateCoin trong phần AddWin đang thấy; vì vậy không đồng nhất tăng tiến độ với tăng coin. Hình 18 có lời gọi gián tiếp sau khi cập nhật; chức năng đầy đủ của đích gọi đó chưa được khôi phục.

6.3. Khoản thưởng 500 coin



Hình 19. ReceiveReward gọi StateCoin với lượng 500 coin.

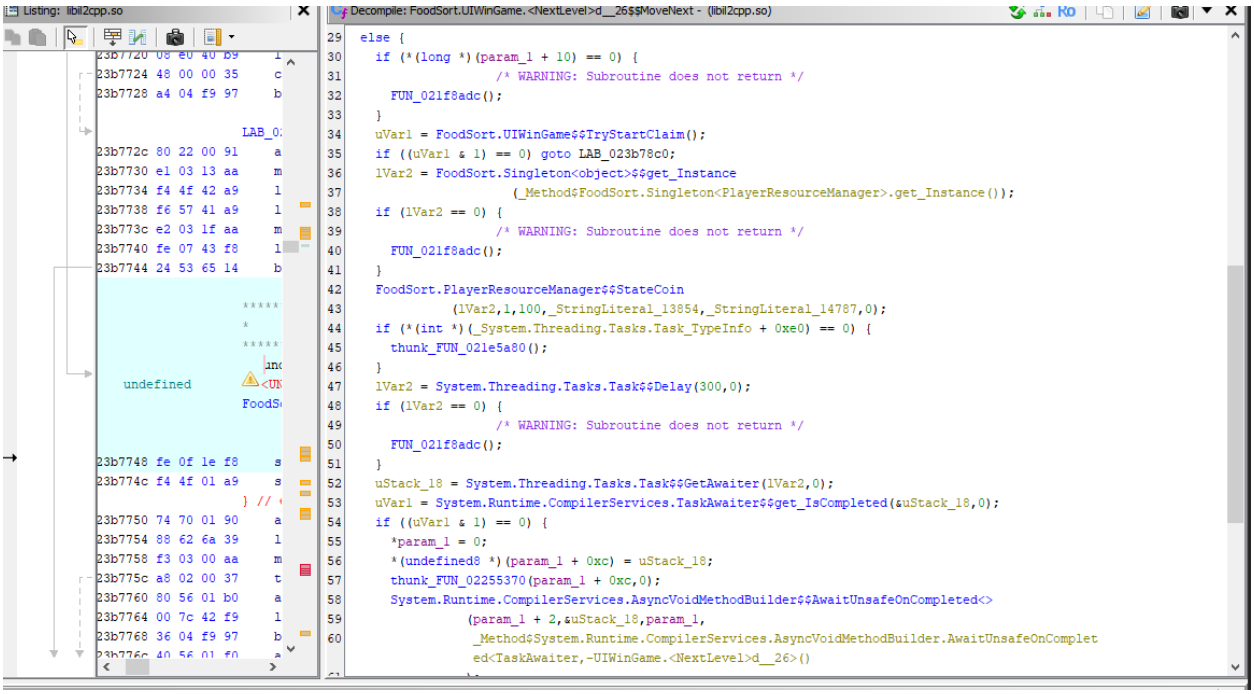
ReceiveReward tại 0x023A2DDC gọi StateCoin với state = true và lượng 500. Kết hợp với phép chọn dấu trong mục 5.3, đây là thao tác cộng 500 vào số dư hiện tại. Sau đó, mã đặt cờ tại offset 0x78 thành 1 và gọi SetRewardButtonInteractable(false).

Khoản này thuộc nhánh RewardManager đã khảo sát, khác với thưởng thường trong UIWinGame.NextLevel. Dữ liệu hiện có chưa nối đầy đủ mọi điều kiện mở giao diện ReceiveReward, nên không kết luận mỗi lần thắng level tự động cộng 500. Phần này giúp loại bỏ một kết luận ban đầu dễ nhầm khi chỉ thấy hằng số 500.

7. Thưởng thường trong UIWinGame.NextLevel

7.1. Xác định phần xử lý bất đồng bộ

NextLevel là phương thức bất đồng bộ. Logic chính nằm trong UIWinGame.<NextLevel>d__26.MoveNext tại 0x023B7748. Ở nhánh bắt đầu, hàm gọi TryStartClaim; chỉ khi kết quả cho phép tiếp tục, chương trình lấy PlayerResourceManager và gọi StateCoin.

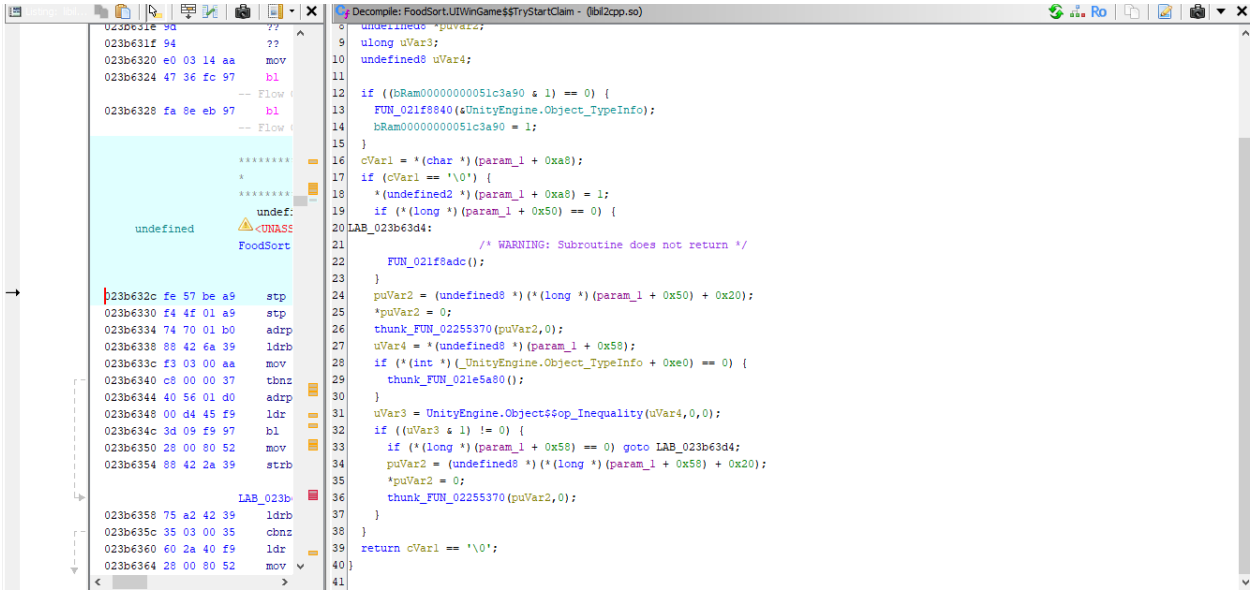


Hình 20. NextLevel.MoveNext gọi StateCoin với lượng 100 coin.

Lời gọi ở Hình 20 truyền state = true, coin = 100. Do StateCoin chọn dấu dương và lưu số dư như mục 5.3, nhánh này cộng 100 vào coin hiện có. Đây là bằng chứng cho luồng thưởng thường trên màn hình thắng, không phải đặt tổng số dư về 100.

Luồng được xác định là màn hình thắng/NextLevel → MoveNext → TryStartClaim → StateCoin → trường coin 0x20 → PlayerPrefs. Kết luận áp dụng cho nhánh đã đọc; các loại thưởng khác phải được kiểm tra bằng lời gọi riêng. 100 ở đây là lượng tăng, khác vai trò của mặc định 100 trong Start.

7.2. Điều kiện bắt đầu thao tác nhận thưởng



Hình 21. TryStartClaim kiểm tra cờ và trả kết quả cho phép bắt đầu.

TryStartClaim tại 0x023B632C đọc byte ở offset 0xA8. Nếu byte bằng 0, hàm ghi giá trị 1 vào vùng 2 byte bắt đầu tại offset này, xử lý các đối tượng nút và trả về true. Nếu byte ban đầu khác 0, phần xử lý bị bỏ qua và hàm trả về false. Dòng return dùng giá trị cờ đã đọc trước khi ghi.



```
        [FieldOffset(Offset = "0x50")]
        [SerializeField]
        private UIClick _uIClickNextLevel;

        // Token: 0x04000BD3 RID: 3027
        [Token(Token = "0x4000BD3")]
        [FieldOffset(Offset = "0x58")]
        [SerializeField]
        private UIClick _uIClickRewardX2;

        // Token: 0x04000BD4 RID: 3028
        [Token(Token = "0x4000BD4")]
        [FieldOffset(Offset = "0x60")]
        [SerializeField]
        private Transform _victoryImg;

        // Token: 0x04000BD5 RID: 3029
        [Token(Token = "0x4000BD5")]
        [FieldOffset(Offset = "0x68")]
        [SerializeField]
        private Transform _buttonContinue;

        // Token: 0x04000BD6 RID: 3030
        [Token(Token = "0x4000BD6")]
        [FieldOffset(Offset = "0x70")]
        [SerializeField]
        private Transform _buttonRewardX2;

        // Token: 0x04000BD7 RID: 3031
        [Token(Token = "0x4000BD7")]
        [FieldOffset(Offset = "0x78")]
        [SerializeField]
        private Transform _coinHeader;

        // Token: 0x04000BD8 RID: 3032
        [Token(Token = "0x4000BD8")]
        [FieldOffset(Offset = "0x80")]
        [SerializeField]
        private Transform _coinGain;

        // Token: 0x04000BD9 RID: 3033
        [Token(Token = "0x4000BD9")]
        [FieldOffset(Offset = "0x88")]
        [SerializeField]
        private ParticleSystem _VFXconfetti;

        // Token: 0x04000BDA RID: 3034
        [Token(Token = "0x4000BDA")]
        [FieldOffset(Offset = "0x90")]
        [SerializeField]
        private ParticleSystem _VFXcoin;

        // Token: 0x04000BDB RID: 3035
        [Token(Token = "0x4000BDB")]
        [FieldOffset(Offset = "0x98")]
        private Tween _tween;

        // Token: 0x04000BDC RID: 3036
        [Token(Token = "0x4000BDC")]
        [FieldOffset(Offset = "0xA0")]
        private Tween _tweenRawImage;

        // Token: 0x04000BDD RID: 3037
        [Token(Token = "0x4000BDD")]
        [FieldOffset(Offset = "0xA8")]
        private bool _isClaimingReward;

        // Token: 0x04000BDE RID: 3038
        [Token(Token = "0x4000BDE")]
        [FieldOffset(Offset = "0xA9")]
        private bool _isRewardAdCallbackHandled;

        // Token: 0x04000BDF RID: 3039
        [Token(Token = "0x4000BDF")]
        [FieldOffset(Offset = "0xB0")]
        private Tween _tweenScale;
    }
}
```

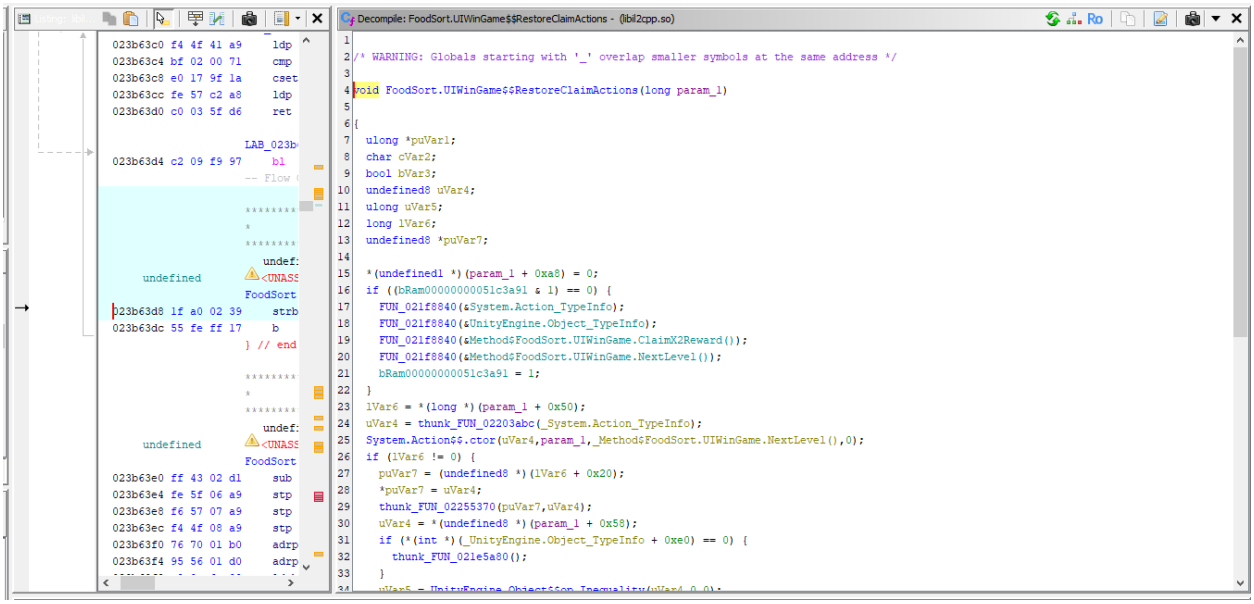
Activate Windows
Go to Settings to activate Windows.

Hình 22. Hai cờ nhận thưởng tại offset 0xA8 và 0xA9 trong dnSpy.

Đối chiếu dnSpy: 0xA8 là bool _isClaimingReward; 0xA9 là bool _isRewardAdCallbackHandled. Trên thư viện ARM64 đang phân tích, lệnh ghi 2 byte giá trị 1 tương ứng đặt cờ 0xA8 = true và cờ 0xA9 = false. Tên trường cùng offset làm rõ ý nghĩa thao tác mà mã giả chỉ biểu diễn là ghi vùng nhớ.

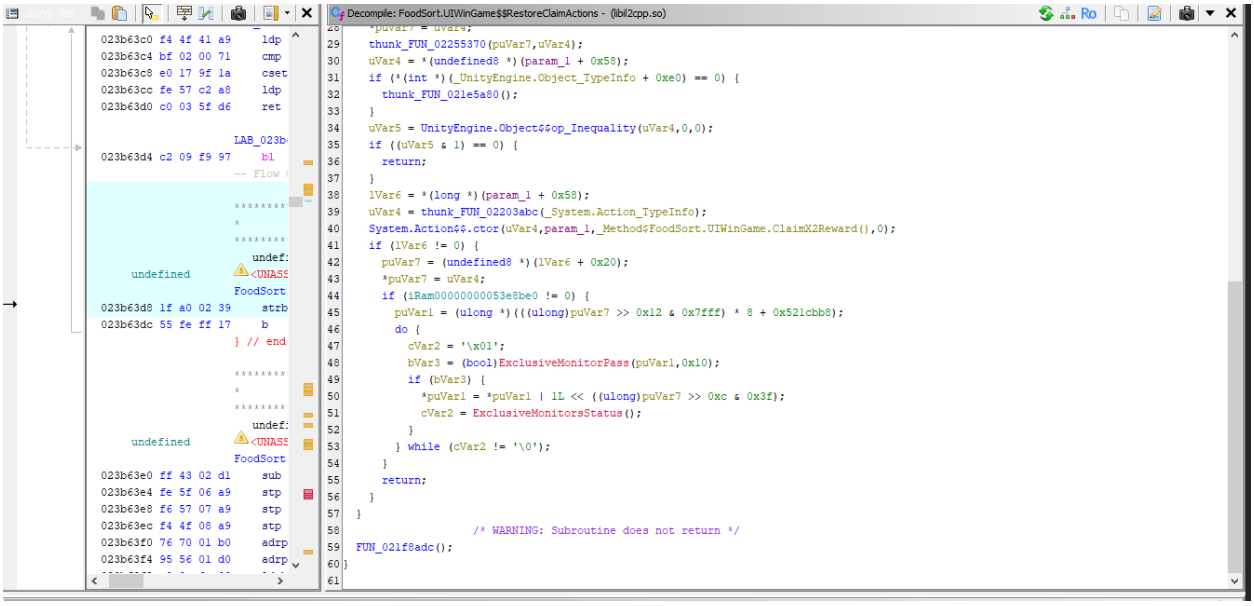
Hai cờ đảm nhiệm việc kiểm soát bắt đầu nhận thưởng và callback quảng cáo. Chúng không phải dữ liệu số dư, cũng không chứng minh một cơ chế chống sửa tài nguyên trên máy chủ.

7.3. Đặt lại cờ và khôi phục các nút



Hình 23. RestoreClaimActions đặt lại cờ và khôi phục nút NextLevel.

RestoreClaimActions tại 0x023B63D8 ghi 0 vào byte ở offset 0xA8. Phần xử lý tiếp theo tạo delegate NextLevel và gắn vào đối tượng _uiClickNextLevel tại 0x50. Listing cho thấy hàm chuyển tiếp tới BindClaimActions; Decompiler trình bày phần khôi phục hành động sau thao tác đặt lại cờ.



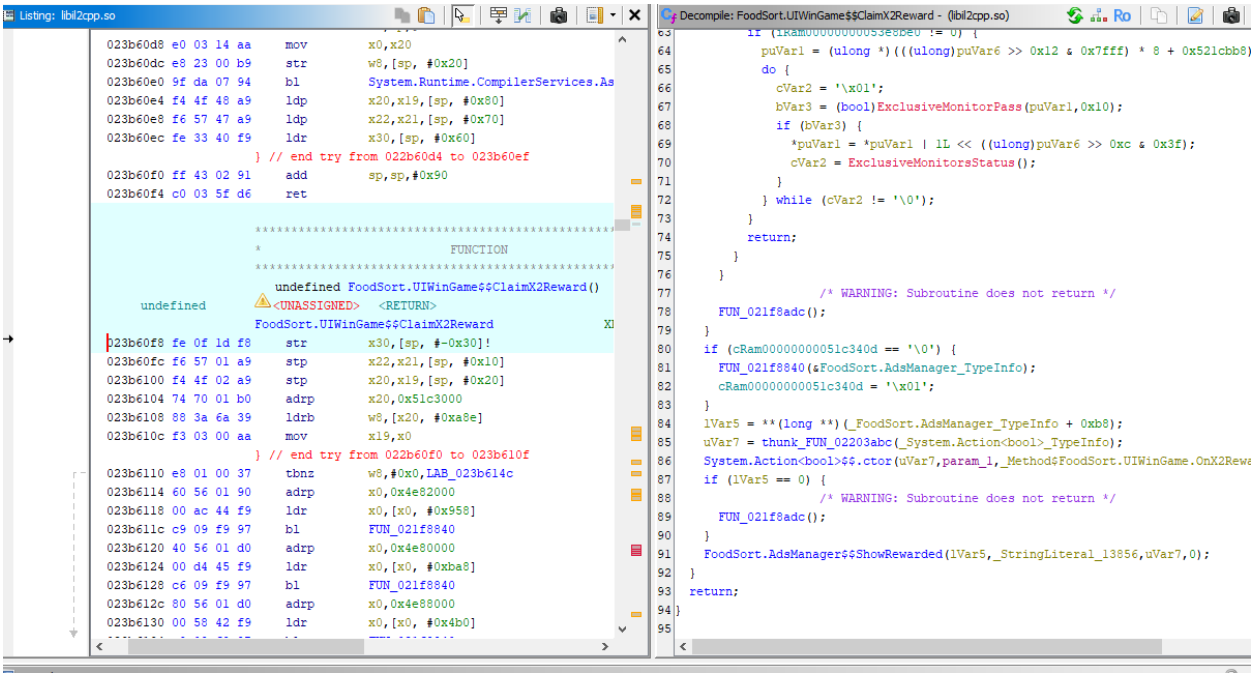
Hình 24. RestoreClaimActions khôi phục callback nút X2.

Khi _uiClickRewardX2 tại offset 0x58 tồn tại, callback ClaimX2Reward được gắn lại. Như vậy, cơ chế có thể mở lại thao tác nhận thưởng sau khi cờ được đặt về false. Không kết luận cờ _isClaimingReward giới hạn nhận thưởng một lần vĩnh viễn; cần xét vòng đời giao diện và từng nhánh gọi khôi phục.

8. Luồng quảng cáo nhận thưởng X2

8.1. Gọi quảng cáo và đăng ký callback

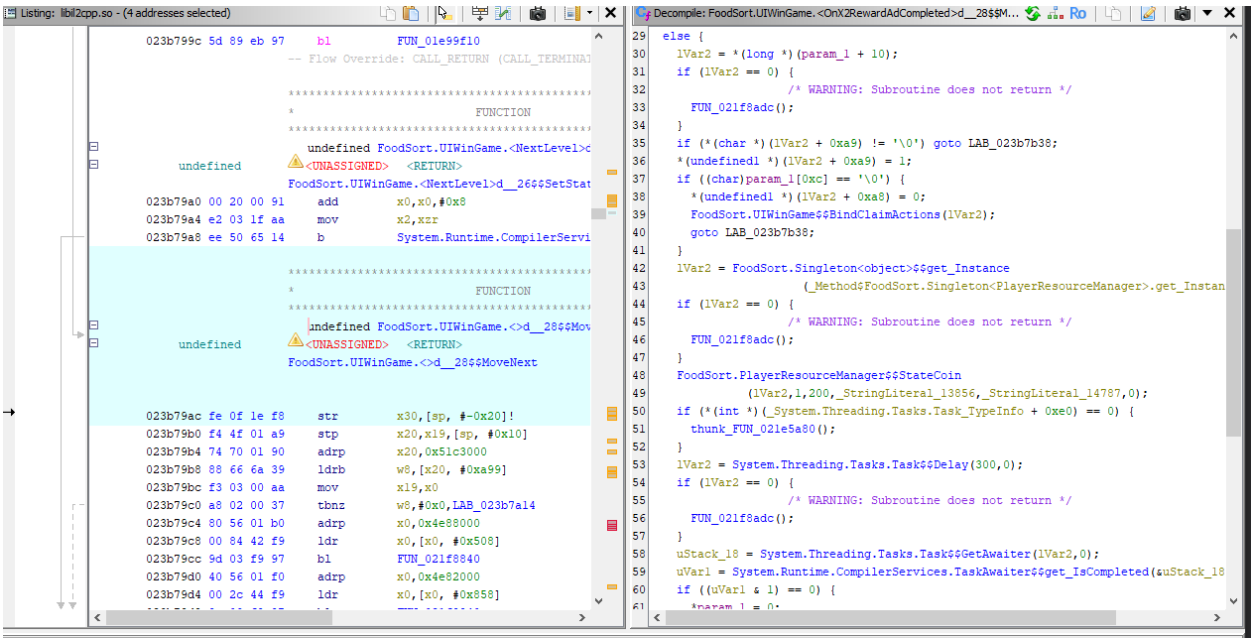
ClaimX2Reward tại 0x023B60F8 bắt đầu bằng TryStartClaim. Nếu không được phép tiếp tục, hàm không đi vào nhánh quảng cáo. Nếu AdsManager không tồn tại, mã đã khảo sát đặt _isClaimingReward về false và khôi phục callback hai nút. Nếu AdsManager tồn tại, chương trình tạo Action<bool> trỏ đến OnX2RewardAdCompleted.



Hình 25. ClaimX2Reward đăng ký callback và gọi ShowRewarded.

Lời gọi ShowRewarded nhận đối tượng AdsManager, ký hiệu chuỗi placement và callback kết quả. Đây là điểm nối từ nút X2 sang xử lý quảng cáo; việc cộng coin nằm trong callback, không phải ở chính lời gọi ShowRewarded. Tên thật của chuỗi placement chưa được giải mã.

8.2. Xử lý kết quả và ngăn callback lặp

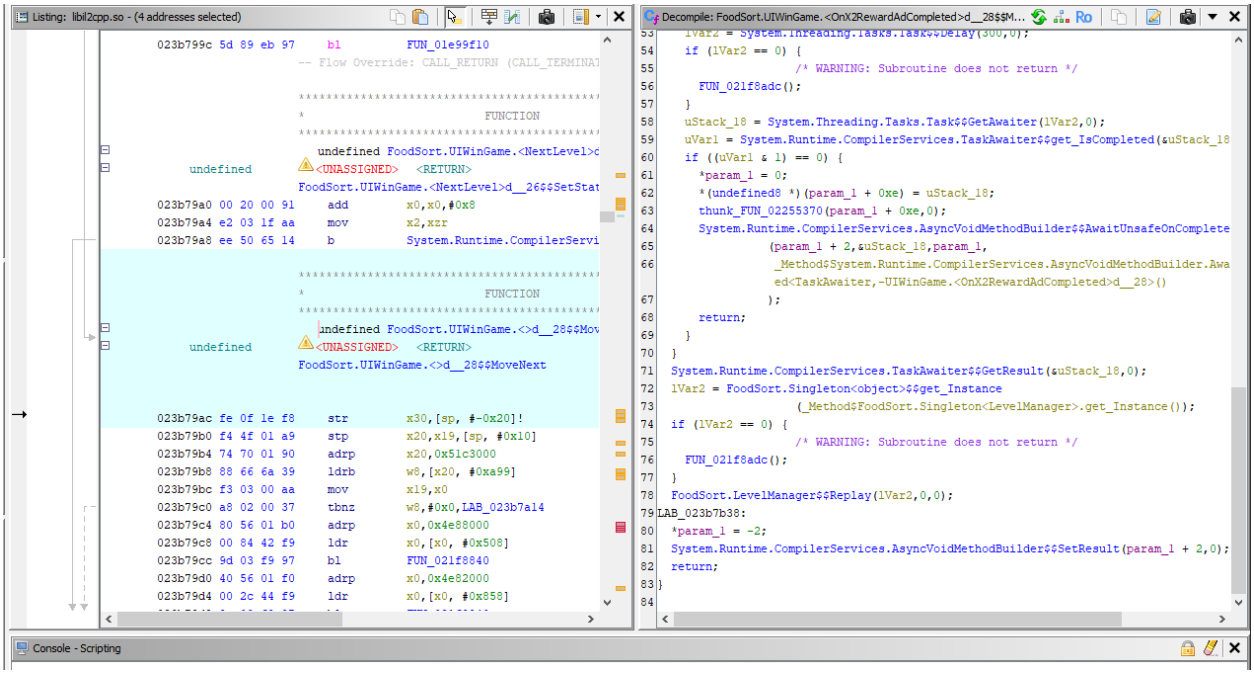


Hình 26. Callback quảng cáo kiểm soát xử lý lặp và cộng 200 coin.

Logic bất đồng bộ của OnX2RewardAdCompleted nằm trong MoveNext tại 0x023B79AC. Hàm kiểm tra _isRewardAdCallbackHandled ở 0xA9; nếu đã đặt thì bỏ qua xử lý, nếu chưa thì đặt thành true trước khi kiểm tra receivedReward.

Nếu receivedReward = false, mã đặt _isClaimingReward về false, gọi BindClaimActions rồi đi tới nhánh hoàn tất mà không cộng coin. Nếu receivedReward = true, StateCoin được gọi với state = true, coin = 200. Mức 200 gấp đôi lượng 100 trong NextLevel; chưa có bằng chứng cộng gộp hai nhánh thành 300.

8.3. Thao tác sau khi cộng thưởng



Hình 27. Nhánh X2 chờ 300 ms, gọi Replay và hoàn tất tác vụ.

Nhánh thành công gọi Task.Delay(300), lưu trạng thái chờ nếu tác vụ chưa hoàn tất, rồi tiếp tục sau GetResult. Sau đó, chương trình lấy LevelManager và gọi Replay; cuối cùng đặt trạng thái tác vụ thành -2, gọi SetResult và trả về. Thứ tự quan sát được là cộng, lưu coin → chờ → gọi Replay → hoàn tất.

Báo cáo xác nhận lời gọi Replay, chưa xác định đầy đủ hành vi của phương thức này; không tự đổi kết luận thành chuyển sang level mới hoặc chơi lại level hiện tại chỉ từ tên hàm.

9. Tổng hợp kết quả và nhận xét an toàn

9.1. Phân biệt các giá trị coin

Giá trị mặc định 100 xuất hiện khi Start đọc dữ liệu chưa có; thưởng thường +100 xuất hiện trong NextLevel; thưởng X2 +200 chỉ đi qua callback báo thành công; +500 thuộc ReceiveReward riêng. Các giá trị này có điều kiện và vai trò khác nhau. Phép cập nhật chung do StateCoin thực hiện và cùng dùng trường số dư ở 0x20.

9.2. Kịch bản can thiệp và tác động

Một kịch bản cần kiểm chứng là lượng coin truyền vào StateCoin hoặc dữ liệu lưu cục bộ bị thay đổi phía client. Nếu đường đi đó thực thi mà không có kiểm tra độc lập, số dư có thể lệch quy tắc, làm sai tài nguyên người chơi hoặc tiến trình mua, sử dụng vật phẩm. Cơ sở kỹ thuật là hằng số thưởng truyền từ client và thao tác đọc, ghi PlayerPrefs đã quan sát.

Đây là nhận định về vị trí có thể cần kiểm tra, không phải kết quả can thiệp thành công. Hai cờ giao diện giúp tránh thao tác/callback lặp nhưng không bảo đảm tính toàn vẹn của số dư khi client bị sửa.

9.3. Giới hạn của phân tích và cách kiểm chứng

Phân tích tĩnh không chứng minh mọi nhánh đều thực thi trong thực tế hoặc game có hay không có đồng bộ máy chủ. Chưa giải mã các StringLiteral; chưa khảo sát toàn bộ vòng đời màn hình thắng, đích gọi gián tiếp, nội dung Replay và điều kiện mở giao diện thưởng 500. Các hằng số, địa chỉ và nhánh được kết luận trong phạm vi bộ ảnh đã đọc.

Có thể kiểm chứng bằng các tình huống: dữ liệu mới khi mở game; mở lại với dữ liệu đã lưu; nhận thưởng thường; quảng cáo thành công; quảng cáo không nhận thưởng; callback hoặc thao tác lặp. Mỗi tình huống cần ghi số dư trước/sau, trạng thái dữ liệu, kết quả quảng cáo và điều kiện giao diện. Kết quả chạy phải được báo cáo riêng; các phép thử này chưa được thực hiện ở đây.

9.4. Hướng giảm phụ thuộc vào client

Với hệ thống có máy chủ, có thể để máy chủ xác minh kết quả level, tính lượng thưởng theo quy tắc và cập nhật số dư; mỗi yêu cầu nhận thưởng có mã giao dịch để xử lý một lần. Nếu nền tảng quảng cáo hỗ trợ xác minh kết quả trên máy chủ, có thể dùng kết quả đã xác minh thay cho một cờ bool do client tự quyết định. Dữ liệu client chỉ phục vụ hiển thị hoặc bộ nhớ đệm. Đây là đề xuất thiết kế, không mô tả kiến trúc đã xác nhận của SandSort.

10. Kết luận và hồ sơ nộp

Bài thực hành đã thu được hình ảnh bottle 1, tạo dữ liệu tra cứu IL2CPP và đối chiếu mã ARM64 để nối nguồn dữ liệu coin với trường số dư và thao tác lưu. Hai mục tiêu chính được xác định: Start nạp PlayerPrefs với mặc định 100 vào offset 0x20; nhánh NextLevel cộng 100 qua StateCoin. Khảo sát bổ sung xác định X2 cộng 200, ReceiveReward cộng 500 và cơ chế đặt/khôi phục các cờ nhận thưởng.

Báo cáo sử dụng 27 ảnh, mỗi nhóm ảnh phục vụ một kết luận riêng. Cần bổ sung lớp, tên/phiên bản hoặc mã nhận diện APK; ảnh Preview và loại asset trong UABEA nếu giảng viên yêu cầu đủ minh chứng trực quan. Hồ sơ kết quả cần giữ PNG, dump.cs, DummyDll, script.json, ghidra.py, ảnh thực hành và project Ghidra theo hướng dẫn GVTH. Các thông tin chưa có được ghi rõ thay vì suy đoán.

Phụ lục. Các địa chỉ chính đã kiểm chứng

Bảng sau dùng để tra cứu nhanh. RVA và địa chỉ Ghidra được tách riêng; Image Base của project là 0x100000. Các địa chỉ chỉ áp dụng cho thư viện đã phân tích.

Phương thức	RVA	Địa chỉ Ghidra
PlayerResourceManager.Start	0x22A15D4	0x023A15D4
PlayerResourceManager.StateCoin	0x22A16F8	0x023A16F8
UIWinGame.NextLevel.MoveNext	0x22B7748	0x023B7748
UIWinGame.TryStartClaim	0x22B632C	0x023B632C
UIWinGame.RestoreClaimActions	0x22B63D8	0x023B63D8
UIWinGame.ClaimX2Reward	0x22B60F8	0x023B60F8
UIWinGame.OnX2RewardAdCompleted.MoveNext	0x22B79AC	0x023B79AC
RewardManager.ReceiveReward	0x22A2DDC	0x023A2DDC